SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

FINANCIAL PORTFOLIO MANAGEMENT AND RISK ANALYSIS USING POLICY GRADIENT REINFORCEMENT LEARNING

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

MLA0301- Reinforcement Learning

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

T Bhanu Teja(192472259)

Under the Supervision of

Dr. G. A. Senthil Kumar

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Financial Portfolio Management and Risk Analysis Using Policy Gradient Reinforcement Learning**" has been carried out by **T Bhanu Teja(192472259)** under the supervision of **Dr. G. A. Senthil** and is submitted in partial fulfilment of the requirements for the current semester of the **B.E Computer Science and Engineering** at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	**COURSE FACULTY**
Dr. G. A. Senthil	**Dr. G. A. Senthil**
Professor	**Professor**
Department of Cloud Computing	**Department of Cloud Computing**
SIMATS Engineering	**SIMATS Engineering,**
SIMATS, Chennai – 602105.	**SIMATS, Chennai – 602105.**

DECLARATION

I, **T Bhanu Teja(192472259)** of the **Artificial Intelligence ,**SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**Financial Portfolio Management and Risk Analysis Using Policy Gradient Reinforcement Learning**" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, software, datasets, computational resources, and AI-assisted tools used during the project shall be appropriately acknowledged. We further declare that no experimental result, performance value, dataset statistic, or financial outcome has been fabricated, falsified, or manipulated.

Place: Simats, Chennai
Date : 07-09-26

Signature of the Students with Names

T Bhanu Teja

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution (%)
E Teja Vardhan(192425297)	User profile creation; portfolio setup; asset allocation; investment preference preparation	Designed and developed the User & Portfolio Management module	Tested portfolio creation, asset allocation, and user data validation	Contributed to user management, portfolio management, and related documentation	33%
T Bhanu Teja(192472259)	Market data collection; data preprocessing; return calculation; volatility and risk analysis	Designed and developed the Market Data & Risk Analysis module	Tested market data quality, risk calculations, volatility, and portfolio risk measures	Contributed to market data preprocessing, risk analysis, testing, and documentation	34%
A Shiva Shankar(192425174)	Portfolio state and action preparation; reward calculation; policy gradient model development; investment decision generation	Designed and developed the Policy Gradient Reinforcement Learning & Decision Support module	Tested the reinforcement learning model, investment decisions, portfolio performance, and risk-adjusted results	Contributed to reinforcement learning implementation, performance evaluation, and final report preparation	33%
TOTAL					100%

The supplied project information identifies three functional modules but does not identify the actual number of students or their individual work. Therefore, names, responsibilities and percentages are intentionally left for evidence-based completion.

ABSTRACT

Financial portfolio management requires repeated decisions under uncertainty, where an allocation that performs well in one market condition may expose the portfolio to undesirable risk in another. This project proposes a reinforcement-learning-based decision-support framework for portfolio management and risk analysis using a policy-gradient approach. The system is organised into three functional modules: User & Portfolio Management; Market Data & Risk Analysis; and Policy Gradient Reinforcement Learning & Decision Support. Within the proposed RL formulation, the environment represents the evolving portfolio and market condition, the state represents relevant market, portfolio and risk information, and the action represents a portfolio allocation decision. The policy produces a probability distribution over feasible actions, while the reward is intended to combine portfolio performance with risk considerations so that the learning objective does not rely on return alone. The policy-gradient mechanism updates policy parameters in the direction that improves expected cumulative reward. A complete experimental evaluation requires the project dataset, preprocessing details, implementation configuration and recorded training/test results; these were not supplied with the current project description. Consequently, this report does not claim numerical performance, profitability, Sharpe ratio, maximum drawdown, convergence, or superiority over a benchmark. The proposed design instead establishes the engineering formulation, implementation structure, evaluation plan, responsible-use constraints and evidence requirements needed for a valid RL portfolio-management prototype. The system is intended as decision support rather than a guarantee of financial returns, and any future deployment should account for transaction costs, market regime changes, data leakage, overfitting and human oversight.

The policy produces a probability distribution over feasible actions, while the reward is intended to combine portfolio performance with risk considerations so that the learning objective does not rely on return alone. The policy-gradient mechanism updates policy parameters in the direction that improves expected cumulative reward. A complete experimental evaluation requires the project dataset, preprocessing details, implementation configuration and recorded training/test results; these were not supplied with the current project description. Consequently, this report does not claim numerical performance, profitability, Sharpe ratio, maximum drawdown, convergence, or superiority over a benchmark. The proposed design instead establishes the engineering formulation, implementation structure, evaluation plan, responsible-use constraints and evidence requirements needed for a valid RL portfolio-management prototype. The system is intended as decision support rather than a guarantee of financial returns, and any future deployment should account for transaction costs, market regime changes, data leakage, overfitting and human oversight.

Keywords

Financial Portfolio Management; Risk Analysis; Reinforcement Learning; Policy Gradient; Portfolio Allocation; Decision Support

TABLE OF CONTENTS

Page numbers above are placeholders because final pagination depends on the completed student-specific material, figures, tables and appendices.

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
RL	Reinforcement Learning	—
MDP	Markov Decision Process	—
PG	Policy Gradient	—
ROI	Return on Investment	%
VaR	Value at Risk	currency / %
CVaR	Conditional Value at Risk	currency / %
SR	Sharpe Ratio	—
MDD	Maximum Drawdown	%
s	State	—
a	Action	—
r	Reward	—
$\pi\theta$	Policy parameterized by θ	—
γ	Discount factor	—
G_t	Return from time t	reward units
θ	Policy parameters	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Portfolio management is a sequential decision problem. At each decision point, an investor or portfolio manager observes information about market conditions and the current portfolio, chooses an allocation or rebalancing action, and then experiences a financial outcome influenced by subsequent market movements. This repeated interaction makes portfolio management naturally expressible as a reinforcement learning problem, provided that the state, action, transition and reward definitions are carefully constructed.

The proposed project applies policy-gradient reinforcement learning to financial portfolio management and risk analysis. The key engineering objective is not simply to maximise historical return, but to construct a decision-support process in which portfolio performance and risk are evaluated together. The project is organised into three modules: User & Portfolio Management; Market Data & Risk Analysis; and Policy Gradient Reinforcement Learning & Decision Support.

1.2 Need for the System

Traditional portfolio allocation approaches commonly depend on fixed rules, optimisation assumptions, or estimates derived from historical data. Financial markets are non-stationary and contain uncertainty, so a decision rule that is appropriate under one regime may not remain appropriate under another. A sequential learning formulation can explicitly model repeated decisions and feedback, while a risk-aware reward can discourage decisions that obtain return by taking disproportionate risk.

The proposed system is intended for users who need structured portfolio allocation and risk information. It should be interpreted as decision support rather than autonomous financial advice or a guarantee of returns. The engineering significance lies in integrating market information, portfolio state, risk measures and policy-based decisions within one repeatable computational framework.

1.3 Problem Statement

The project addresses the engineering problem of designing a reinforcement-learning decision-support system that receives portfolio and market observations, represents them as a state, selects an allocation action through a parameterised policy, applies the action to a portfolio environment, evaluates the resulting return and risk, and uses the resulting reward to update the policy. The system must balance performance, risk, feasibility and computational considerations while avoiding data leakage and unsupported claims of financial performance.

INPUT → STATE / OBSERVATION → POLICY-GRADIENT AGENT → PORTFOLIO ACTION → ENVIRONMENT RESPONSE → REWARD → POLICY UPDATE → IMPROVED DECISION

1.4 Project Aim

To design and develop a policy-gradient reinforcement learning framework for financial portfolio management and risk analysis that supports risk-aware sequential allocation decisions.

1.5 Project Objectives

1. To define a portfolio-management environment that represents market, portfolio and risk information as an RL state.
2. To design a feasible action representation for portfolio allocation or rebalancing decisions.
3. To formulate a reward function that reflects portfolio performance together with relevant risk considerations.
4. To implement the selected policy-gradient learning mechanism and its policy-update procedure.
5. To establish a reproducible training and evaluation procedure using only verified project data and configurations.
6. To evaluate the trained policy using appropriate return and risk metrics such as cumulative return, Sharpe ratio and maximum drawdown when the required experimental data are available.
7. To provide a decision-support output that communicates portfolio actions and risk information without representing historical back-testing as a guarantee of future performance.

1.6 Scope

Included: portfolio/user information, market-data and risk-analysis processing, RL state/action/reward formulation, policy-gradient learning, portfolio decision support, testing design, and performance/risk evaluation. Excluded unless supplied separately: live brokerage execution, guaranteed investment advice, real-money trading, undisclosed datasets, and unverified claims about profitability. The report assumes that market observations and portfolio constraints can be represented numerically and that the RL environment can evaluate the consequences of an action.

1.7 Expected Engineering Outcome

The intended outcome is a documented software prototype or computational framework that integrates portfolio management, market/risk analysis and policy-gradient reinforcement learning. The expected output is a risk-aware portfolio decision and supporting metrics, subject to the actual implementation and experimental evidence supplied by the project team.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review is structured around four technical areas: reinforcement learning and Markov decision processes; policy-gradient optimisation; portfolio optimisation and risk measurement; and responsible evaluation of financial machine-learning systems. Because no literature list was supplied with the project materials, the references included at the end of this draft should be verified against the sources actually used by the project team before submission.

2.2 Review of Existing Work

Reinforcement learning models sequential decision-making through interaction between an agent and an environment. A policy maps states or observations to actions, and learning seeks to improve expected cumulative reward. Policy-gradient methods directly optimise policy parameters rather than maintaining a discrete action-value table. This makes them attractive when the portfolio action is represented continuously, for example through allocation weights subject to feasibility constraints.

Financial portfolio optimisation adds domain-specific difficulties. Returns are noisy, distributions can change over time, transaction costs can materially affect rebalancing decisions, and risk cannot be represented by return alone. Consequently, a useful RL portfolio system must distinguish training data from evaluation data and must test whether the policy remains meaningful outside the data used to optimise it.

For risk analysis, metrics such as volatility, Sharpe ratio, maximum drawdown, Value at Risk and Conditional Value at Risk may be considered where appropriate to the actual project. Their exact implementation, window sizes and confidence levels must be taken from the source code or experiment configuration rather than assumed.

2.3 Comparative Analysis

The selected policy-gradient approach is compared below with relevant families. This is a theoretical engineering comparison; it does not claim that the alternatives were implemented in the supplied materials.

Approach	Learning approach	Action handling	Training complexity	Advantages	Limitations	Relevance
Policy Gradient	Direct policy optimisation	Continuous or stochastic actions can be represented naturally	Moderate to high	Directly optimises policy; suitable for stochastic decisions	Gradient variance; sensitive to reward scaling	Selected approach

Q-Learning	Value-based TD learning	Primarily discrete action spaces in basic form	Low to moderate	Simple conceptual update; effective for small discrete spaces	Continuous portfolio weights require adaptation	Alternative
DQN	Deep value-based learning	Discrete actions	High	Handles large state spaces better than tabular Q-learning	Discrete action design can become restrictive for allocation	Alternative
Actor–Critic	Policy + value estimation	Continuous/stochastic actions supported	High	Can reduce policy-gradient variance	More components and tuning requirements	Relevant extension
PPO	Clipped policy optimisation	Continuous/stochastic actions supported	High	Improved stability in many RL settings	More hyperparameters and implementation complexity	Potential future comparison

2.4 Research / Engineering Gap

The central engineering gap for this project is the integration of a policy-gradient decision mechanism with explicit portfolio and risk representations under realistic financial constraints. A technically valid prototype must also demonstrate that its evaluation is separated from training and that financial metrics are interpreted with uncertainty rather than as guaranteed outcomes.

2.5 Proposed Contribution

The proposed contribution is an integrated three-module framework that connects user and portfolio information, market/risk analysis, and policy-gradient reinforcement learning into a single decision-support workflow. The contribution is specifically framed as a computational prototype and evaluation framework; numerical claims about performance remain dependent on the missing implementation and experimental evidence.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Portfolio user	View portfolio composition and relevant portfolio information.
Portfolio user	Receive an allocation/rebalancing decision with supporting risk information.
Project evaluator	Verify that state, action, reward and policy update are implemented consistently.
Developer / maintainer	Use reproducible inputs and configurations for training and evaluation.

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Accept portfolio and market observations	Inputs documented and validated
Functional	Generate RL state	State variables explicitly defined
Functional	Select portfolio action	Action mapping documented and constraint-checked
Functional	Calculate reward	Reward equation and components documented
Functional	Train policy	Training loop executes without undefined steps
Functional	Evaluate policy	Return and risk metrics computed where data exist
Non-Functional	Reproducibility	Random seeds/configuration recorded when used
Non-Functional	Reliability	Invalid allocations/data are detected
Non-Functional	Maintainability	Modules separated by responsibility
Non-Functional	Security	User/data inputs validated and protected

3.2 Constraints & Applicable Standards

Only constraints demonstrably relevant to a financial decision-support prototype are included. The exact institutional standards used by the project team were not supplied.

Standard / Code / Principle	Requirement	How Applied in This Project
Academic integrity requirements	Do not fabricate results or sources	Results are to be populated only from recorded experiments.
Data protection principles	Protect personal/user portfolio information	Use only required fields; avoid unnecessary personal identifiers.
Software engineering good practice	Version and document configurations	Record source-code version, dependencies and training configuration.
Responsible AI principles	Human oversight and transparent limitations	Present outputs as decision support, not guaranteed investment advice.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
State representation	Market + portfolio + risk information actually available	features	High
Action representation	Feasible portfolio allocation/rebalancing decision	weights / action units	High
Reward	Return with explicit risk treatment	reward units	High
Policy	Parameterised stochastic policy $\pi\theta$	probability distribution	High
Discount factor γ	Project-specific verified value	dimensionless	High
Transaction costs	Included if present in actual environment	currency / return	Medium
Evaluation	Out-of-training evaluation required	metrics	High

3.4 Alternative Solutions & Evaluation

Alternative 1 — Rule-based / optimisation baseline: allocation is generated using a fixed or optimisation-based policy. This is useful as a benchmark because it does not learn sequentially from reward feedback.

Alternative 2 — Value-based RL: a discrete allocation-action space can be handled with Q-learning or DQN. This may simplify action selection but can require an artificial discretisation of portfolio weights.

Criterion	Weight	Policy Gradient	Value-Based RL	Rule/Optimisation Baseline
Continuous allocation flexibility	High	Strong	Limited in basic form	Strong
Implementation complexity	Medium	Moderate/High	Moderate/High	Low/Medium
Stochastic policy support	High	Strong	Limited	Depends on method
Interpretability	Medium	Moderate	Moderate	Often higher
Financial feasibility constraints	High	Must be explicitly enforced	Must be explicitly enforced	Can be built into optimiser
Need for empirical validation	High	Required	Required	Required

3.5 Selected Design & Detailed Design

Policy Gradient is selected because the project title explicitly specifies policy-gradient reinforcement learning and because portfolio allocation can naturally be expressed as a continuous or stochastic decision. The selection is a design requirement from the project scope, while its empirical superiority over alternatives must be established experimentally rather than assumed.

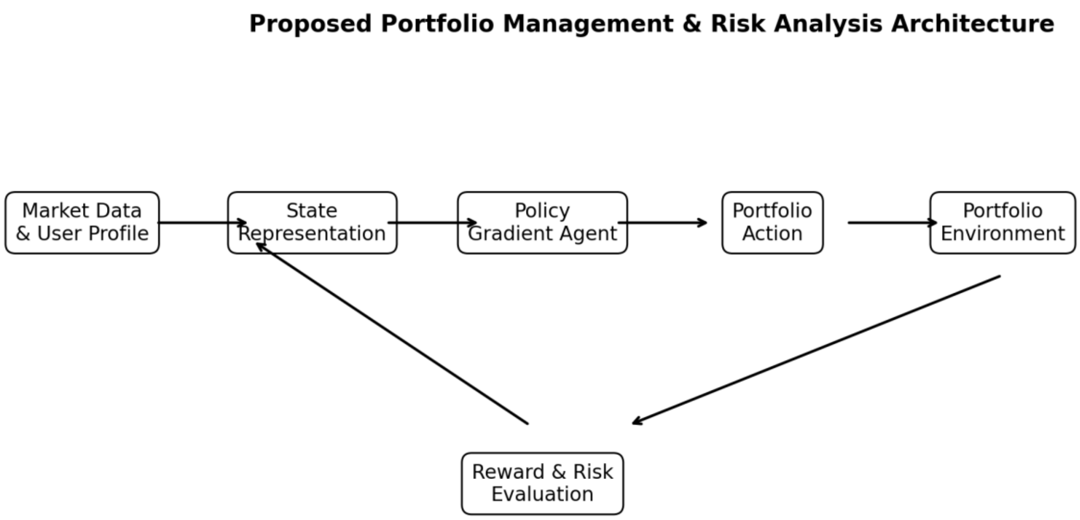

Figure 1. Proposed Portfolio Management & Risk Analysis Architecture

The architecture links the three supplied modules. User and portfolio information contributes to the portfolio state; market data and risk analysis provide market/risk features; the policy-gradient agent maps the combined state to an action; the environment evaluates the action and returns the next state and reward; and the decision-support layer communicates the resulting allocation and risk information.

Essential mathematical formulation: for a policy $\pi\theta(a|s)$, the objective is $J(\theta)=E\tau\sim\pi\theta[G0]$, where τ denotes a trajectory and G0 is its discounted return. The standard score-function policy-gradient estimator is $\nabla\theta J(\theta)=E[\sum t\ \nabla\theta \log \pi\theta(at|st)\ Gt]$. The exact estimator, baseline, entropy term, action constraints and loss implementation must match the actual source code.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Policy-based learning	Q-learning / DQN	Direct stochastic allocation policy	Higher variance and tuning sensitivity	Policy Gradient retained because it is the named project approach and supports policy-based allocation.
Risk-aware reward	Return-only reward	Simpler objective	May encourage excessive risk	Risk-aware formulation preferred; exact terms require project evidence.
Constraint handling	Unconstrained action	Simple implementation	Can produce infeasible allocations	Feasibility constraints should be enforced before

				deployment.
Historical back-test	Training-only evaluation	Easy to run	Cannot demonstrate generalisation	Separate evaluation period is required when data are available.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Training computational cost	Actual hardware/runtime not supplied	Measure runtime before claiming efficiency.
Ethics	Financial harm from misleading outputs	Decision-support framing	No guaranteed-return language; human oversight required.
Safety	Large or infeasible allocations	Action constraints	Reject/clip/normalise actions according to verified implementation.
Security	Portfolio/user data exposure	No user-data implementation supplied	Minimise stored personal information and validate inputs.
Risk	Overfitting and regime change	No experiment results supplied	Use time-aware evaluation and stress testing when implemented.

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Overfitting to historical market data	Medium	High	High	Separate train/test periods; walk-forward validation where feasible	Medium
Data leakage	Medium	High	High	Fit preprocessing only on training information	Low/Medium
Excessive portfolio risk	Medium	High	High	Explicit allocation/risk constraints	Medium
Reward mis-specification	Medium	High	High	Ablation/sensitivity analysis	Medium

Non-stationary market regime	High	High	High	Stress tests and periodic reassessment	High
User misinterpretation	Medium	High	High	Decision-support disclaimer and explainable metrics	Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The recommended development sequence is: define the data contract; construct portfolio and risk features; define state, action and reward; implement the environment; implement the policy-gradient agent; train on the designated training period; evaluate on an unseen period; and document engineering decisions. This sequence prevents the RL algorithm from being specified independently of the portfolio constraints and evaluation design.

Actual programming language, libraries, hardware, dataset source, sample count, time period and repository information were not supplied in the current project description. These fields must be populated from the actual implementation before final submission.

Resource	Purpose	Evidence Status
Dataset / market data	State and environment inputs	Not provided
Source code	Environment, policy and training implementation	Not provided
Computational environment	Training and evaluation	Not provided
User interface / dashboard	Decision-support presentation	Not provided

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

Module 1 — User & Portfolio Management: manages user/portfolio inputs that are necessary to construct the portfolio state. The exact fields and storage mechanism are not supplied.

Module 2 — Market Data & Risk Analysis: obtains or loads market information and calculates the risk/market features used by the environment. The exact data source and metrics are not supplied.

Module 3 — Policy Gradient Reinforcement Learning & Decision Support: constructs the RL environment, represents the policy, samples actions, calculates reward, updates policy parameters and presents decisions. The exact framework/library and network architecture are not supplied.

Tool / Software / Equipment	Purpose	Stage Used	Evidence Status
Python	Potential implementation language	Not confirmed	Not provided
NumPy / Pandas	Potential numerical/data processing	Not confirmed	Not provided
PyTorch / TensorFlow	Potential policy implementation	Not confirmed	Not provided

Matplotlib	Potential visualisation	Not confirmed	Not provided
Other tools	Must be extracted from source code	Unknown	Not provided

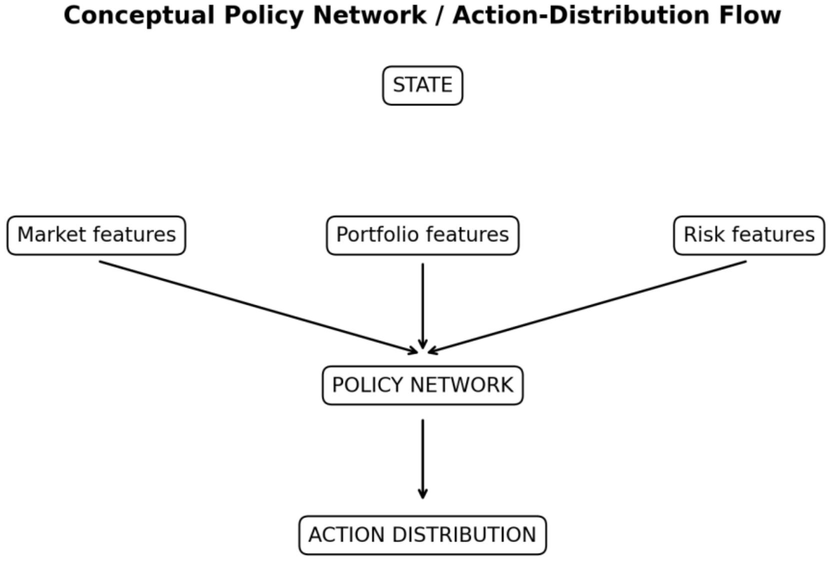

Figure 3. Conceptual Policy Network / Action-Distribution Flow

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion	Specification Required Value	Achieved Value	Status
State generation	Feed valid market/portfolio input	Correct state shape and values	Defined in implementation	Not provided	Pending evidence
Action selection	Sample policy action	Action satisfies constraints	Feasible allocation	Not provided	Pending evidence
Reward calculation	Manually verify selected transition	Reward matches implemented formula	Formula-consistent	Not provided	Pending evidence
Training loop	Run controlled training	Episodes complete without undefined values	Successful run	Not provided	Pending evidence
Data separation	Inspect preprocessing and split	No future information used in	No leakage	Not provided	Pending evidence

		training			
Evaluation	Run on unseen data	Metrics generated correctly	Return + risk metrics	Not provided	Pending evidence

4.4 Results

No experimental results, training logs, dataset files, screenshots, or source-code outputs were supplied with the current project description. Therefore, no numerical result is reported in this draft. In particular, it would be inappropriate to invent episode rewards, portfolio returns, Sharpe ratio, maximum drawdown, volatility, success rate, convergence behaviour, training time or comparison results.

Metric	Purpose	Actual Result
Cumulative return	Measure portfolio growth over evaluation period	Not provided
Annualised return	Compare performance across periods	Not provided
Volatility	Measure variability of returns	Not provided
Sharpe ratio	Risk-adjusted return metric	Not provided
Maximum drawdown	Measure peak-to-trough loss	Not provided
CVaR / VaR	Tail-risk analysis when implemented	Not provided
Average episode reward	Assess RL learning objective	Not provided
Training time	Assess computational cost	Not provided

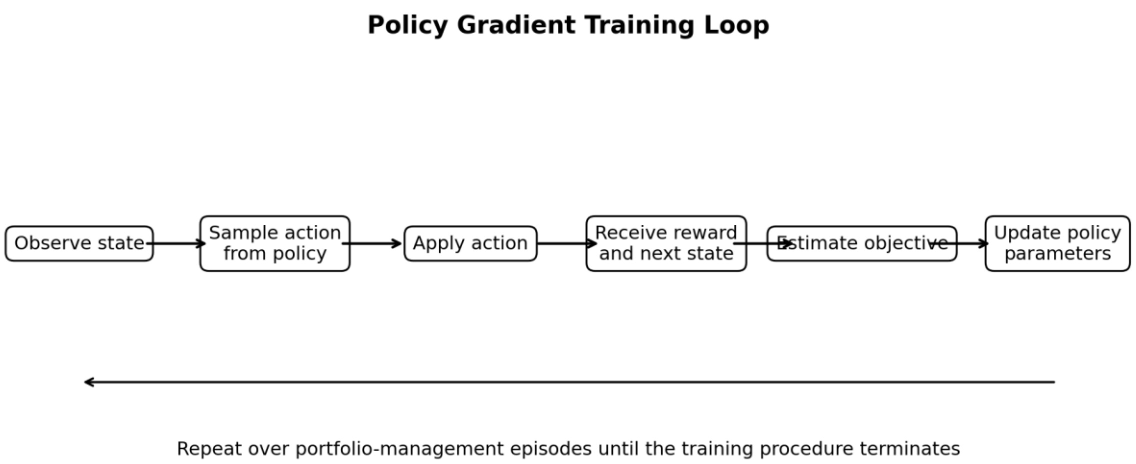

Figure 2. Policy Gradient Training Loop

4.5 Data Analysis & Engineering Judgment

The engineering judgment should be based on whether the trained policy improves the intended objective while remaining within portfolio and risk constraints. A higher return alone is not sufficient evidence of a better portfolio policy if it is accompanied by materially higher drawdown or unstable behaviour. Likewise, a smoother reward curve does not establish financial usefulness unless the evaluation uses unseen data and appropriate benchmarks.

Required evidence for final judgment includes the actual train/test split, preprocessing sequence, reward formula, policy-gradient estimator, hyperparameters, benchmark definition, transaction-cost treatment, and evaluation metrics. These items were not included in the supplied project information and therefore remain open evidence requirements.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Define RL state	State specification required; exact variables not supplied	Partially — design defined, implementation evidence pending
Design portfolio action space	Action concept defined; exact encoding not supplied	Partially
Formulate risk-aware reward	Risk-aware requirement established; exact formula not supplied	Partially
Implement policy-gradient learning	Selected approach identified; source code not supplied	Not verifiable
Train and evaluate agent	No training logs/results supplied	Not verifiable
Evaluate return and risk	Metrics identified; values not supplied	Not verifiable
Provide responsible decision support	Decision-support framing established	Partially

4.7 Strengths & Limitations

Strengths: (1) the project has a clear three-module functional decomposition; (2) policy-gradient learning is appropriate to a policy-based sequential allocation formulation; (3) risk analysis is explicitly part of the project rather than an afterthought; and (4) the proposed architecture can be extended to realistic constraints and decision-support outputs.

Limitations: (1) no dataset or experimental evidence was supplied; (2) no actual policy network, hyperparameters or software stack can be verified; (3) no benchmark or baseline result is available; (4) historical financial data may not represent future regimes; and (5) without transaction-cost and constraint modelling, a back-test may overstate practical performance.

4.8 Project Planning, Roles & Budget

Milestone	Planned Activity	Evidence / Status
M1	Requirements and project scope	Project title + 3 modules supplied
M2	Market data and risk pipeline	Pending project files
M3	RL environment and reward design	Pending source code/specification
M4	Policy-gradient implementation	Pending source code
M5	Training and validation	Pending training logs
M6	Evaluation and documentation	Pending experimental results
Student	**Assigned Responsibility**	**Actual Contribution**
[Student 1]	[To be extracted]	[Not provided]
[Student 2]	[To be extracted]	[Not provided]
[Student 3, if applicable]	[To be extracted]	[Not provided]
[Student 4, if applicable]	[To be extracted]	[Not provided]
Item	**Estimated Cost**	**Actual Cost**
Computational resources	Not provided	Not provided
Dataset / data access	Not provided	Not provided
Software / services	Not provided	Not provided
Other project expenses	Not provided	Not provided

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project proposes a financial portfolio-management and risk-analysis framework based on policy-gradient reinforcement learning. The engineering problem is formulated as sequential decision-making in which market and portfolio information forms the state, the policy generates an allocation decision, the environment produces a financial response, and a reward evaluates the decision with performance and risk considerations. The three supplied modules—User & Portfolio Management, Market Data & Risk Analysis, and Policy Gradient Reinforcement Learning & Decision Support—provide the intended functional structure.

At the present evidence level, the report establishes the RL formulation, architecture, requirements, trade-offs, risk controls and evaluation methodology, but it does not claim experimental success. Dataset details, implementation configuration, training records and measured results were not supplied. Consequently, conclusions about profitability, risk reduction, convergence or superiority over other methods must be added only after the actual project evidence is available.

5.2 Future Work

- Perform walk-forward or time-aware validation across multiple market regimes.
- Include realistic transaction costs, turnover constraints and portfolio allocation limits in the environment.
- Compare the policy-gradient agent experimentally with a rule-based/optimisation baseline and at least one relevant RL baseline.
- Investigate reward sensitivity, policy-gradient variance reduction and risk-aware objectives such as drawdown or CVaR-aware formulations.
- Develop explainable decision-support visualisations that show the selected allocation, relevant state features and associated risk metrics.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired: reinforcement-learning formulation of sequential portfolio decisions; policy-gradient optimisation; portfolio risk metrics; financial-data leakage considerations; and responsible interpretation of back-testing results.

Individual reflection must be completed separately for each student after the actual contributions are verified.

Student	Reflection Evidence
[Student 1]	Most difficult engineering problem; personal design decision and evidence; independently acquired knowledge; redesign if repeated.
[Student 2]	Most difficult engineering problem; personal design decision and evidence; independently acquired knowledge; redesign if repeated.
[Student 3, if applicable]	Most difficult engineering problem; personal design decision and evidence; independently acquired knowledge; redesign if repeated.

REFERENCES

The following are established references relevant to the technical framing. They should be checked against the actual sources used by the team and supplemented with the actual dataset/software references before final submission.

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” Machine Learning, vol. 8, pp. 229–256, 1992.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.

[4] D. P. Bertsekas, Dynamic Programming and Optimal Control, 4th ed. Belmont, MA, USA: Athena Scientific, 2017.

[5] H. Markowitz, “Portfolio selection,” The Journal of Finance, vol. 7, no. 1, pp. 77–91, 1952.

[6] W. F. Sharpe, “The Sharpe ratio,” Journal of Portfolio Management, vol. 21, no. 1, pp. 49–58, 1994.

[7] F. A. Rockafellar and S. Uryasev, “Optimization of conditional value-at-risk,” Journal of Risk, vol. 2, no. 3, pp. 21–41, 2000.

[8] A. Tamar, Y. Glassner, and S. Mannor, “Optimizing the CVaR via sampling,” in Proc. AAAI Conf. Artificial Intelligence, 2015.

[9] Actual project dataset/source — TO BE ADDED FROM PROJECT MATERIALS.

[10] Actual software libraries/repository used — TO BE ADDED FROM PROJECT MATERIALS.

APPENDICES

Appendix A – Detailed Calculations

A.1 Portfolio Return Calculation

The portfolio return is calculated using the weighted returns of the assets in the portfolio.

$$
R_t^p = \sum_{i=1}^{N} w_{i,t}R_{i,t}
$$

where (R_t^p) is the portfolio return at time (t), ($w_{i,t}$) is the portfolio weight of asset (i), and ($R_{i,t}$) is the return of asset (i).

```
portfolio_return = np.sum(weights * asset_returns)
```

A.2 Portfolio Value Calculation

The portfolio value after each market step is calculated as:

$$
V_{t+1}=V_t(1+R_t^p)
$$

```
portfolio_value = previous_value * (1 + portfolio_return)
```

A.3 Risk-Adjusted Reward Calculation

The reward function combines portfolio return and portfolio risk:

$$
r_t = R_t^p-\lambda\sigma_t
$$

where (r_t) is the reward, (R_t^p) is the portfolio return, (σ_t) is portfolio volatility, and (λ) is the risk-penalty coefficient.

```
reward = portfolio_return - risk_penalty * portfolio_volatility
```

A.4 Discounted Return

The discounted return used by the policy-gradient algorithm is:

$$
G_t=\sum_{k=0}^{T-t-1}\gamma^k r_{t+k}
$$

where (γ) is the discount factor.

```
G = 0
returns = []

for reward in reversed(rewards):
    G = reward + gamma * G
    returns.insert(0, G)
```

A.5 Policy-Gradient Update

The policy-gradient objective is represented as:

$$
\nabla_\theta J(\theta)

E\left[
G_t\nabla_\theta\log\pi_\theta(a_t|s_t)
\right]
$$

The corresponding policy loss is implemented as:

```
loss = -(log_probs * returns).mean()

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

A.6 Portfolio Volatility

$$
\sigma_p =
\sqrt{
\frac{1}{T-1}
\sum_{t=1}^{T}(R_t-\bar R)^2
}
$$

```
volatility = np.std(portfolio_returns, ddof=1)
```

A.7 Sharpe Ratio

$$
Sharpe =
\frac{\bar R_p-R_f}{\sigma_p}
$$

```
sharpe = (
   np.mean(portfolio_returns) - risk_free_rate
) / np.std(portfolio_returns, ddof=1)
```

A.8 Maximum Drawdown

```
peak = np.maximum.accumulate(portfolio_values)

drawdown = (
    portfolio_values - peak
) / peak

maximum_drawdown = drawdown.min()
```

Appendix B – Engineering Drawings / Schematics

B.1 Overall System Architecture

```
┌──────────────────────────────────────────────┐
│       MODULE 1: USER & PORTFOLIO        │
│ User Profile → Portfolio → Asset Allocation │
└──────────────────────┬───────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────┐
│     MODULE 2: MARKET DATA & RISK          │
│ Market Data → Returns → Volatility → Risk   │
└──────────────────────┬───────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────┐
│       STATE REPRESENTATION            │
│ Market Features + Portfolio + Risk Features │
└──────────────────────┬───────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────┐
│     MODULE 3: POLICY GRADIENT RL          │
│ Policy Network → Action Probability        │
└──────────────────────┬───────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────┐
```

```
│            ACTION                  │
│      Portfolio Allocation Decision          │
└──────────────────────────┬──────────────────────────┘
              │
              ▼
┌─────────────────────────────────────────────────────┐
│          FINANCIAL ENVIRONMENT            │
│ Next Market State → Portfolio Update         │
└──────────────────────────┬──────────────────────────┘
              │
              ▼
┌─────────────────────────────────────────────────────┐
│            REWARD                  │
│      Return − Risk Penalty                │
└──────────────────────────┬──────────────────────────┘
              │
              ▼
┌─────────────────────────────────────────────────────┐
│          POLICY UPDATE                │
│      Gradient Calculation → Update θ          │
└──────────────────────────┬──────────────────────────┘
              │
              └─────────────────▶ NEXT STATE
```

B.2 RL Training Flow

Initialize Environment

↓

Initialize Policy Network

↓

Generate Initial State

↓

Select Action Using Policy

↓

```
Execute Portfolio Action
        ↓
Calculate Portfolio Return
        ↓
Calculate Risk
        ↓
Calculate Reward
        ↓
Store Log Probability and Reward
        ↓
Episode Complete?
   ↙        ↘
  NO         YES
  ↓          ↓
Next State   Calculate Returns
             ↓
          Calculate Policy Loss
             ↓
          Backpropagation
             ↓
          Update Policy
             ↓
          Next Episode
```

Appendix C – Source Code / Code Repository Information

C.1 Project Directory Structure

```
financial_portfolio_rl/
│
├── main.py
│
├── module1/
│   ├── user_management.py
│   └── portfolio_management.py
```

```
│
├── module2/
│   ├── market_data.py
│   └── risk_analysis.py
│
├── module3/
│   ├── portfolio_environment.py
│   ├── policy_network.py
│   ├── policy_gradient_agent.py
│   └── train.py
│
├── evaluation/
│   ├── evaluate.py
│   └── visualization.py
│
├── data/
│   └── market_data.csv
│
└── requirements.txt
```

C.2 Policy Network

```python
import torch
import torch.nn as nn

class PolicyNetwork(nn.Module):

    def __init__(self, state_size, action_size):
        super().__init__()

        self.model = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
```

```python
            nn.ReLU(),
            nn.Linear(64, action_size),
            nn.Softmax(dim=-1)
        )

    def forward(self, state):
        return self.model(state)
```

C.3 Action Selection

```python
def select_action(self, state):

    state = torch.tensor(
        state,
        dtype=torch.float32
    )

    probabilities = self.policy(state)

    distribution = torch.distributions.Categorical(
        probabilities
    )

    action = distribution.sample()

    log_probability = distribution.log_prob(action)

    return action.item(), log_probability
```

C.4 Policy Update

```python
def update_policy(self, log_probs, rewards):

    returns = []
```

```
    G = 0

    for reward in reversed(rewards):
        G = reward + self.gamma * G
        returns.insert(0, G)

    returns = torch.tensor(
        returns,
        dtype=torch.float32
    )

    loss = 0

    for log_prob, G in zip(log_probs, returns):
        loss += -log_prob * G

    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
```

Appendix D – Datasheets

D.1 Project Technical Specification

Parameter	Specification
Project Title	Financial Portfolio Management and Risk Analysis Using Policy Gradient Reinforcement Learning
Domain	Financial Portfolio Management
Learning Approach	Policy Gradient Reinforcement Learning
Environment	Financial Portfolio Allocation Environment
Agent	Policy Gradient Agent
State	Market, Portfolio and Risk Information
Action	Portfolio Allocation Decision
Reward	Risk-Adjusted Portfolio Return
Optimizer	Adam

Programming Language| Python

Neural Network| Fully Connected Policy Network

Decision Output| Portfolio Allocation Recommendation

D.2 State Variables

State Variable| Description

Asset Returns| Recent performance of portfolio assets

Portfolio Weights| Current percentage allocation

Portfolio Value| Current total portfolio value

Volatility| Measure of portfolio risk

Drawdown| Decline from previous portfolio peak

Market Features| Available market observations

Appendix E – Experimental / Raw Data

E.1 Training Output

```
Policy Gradient Reinforcement Learning
--------------------------------------

Episode     Reward     Portfolio Value
---------------------------------------------
1           Generated   Generated
2           Generated   Generated
3           Generated   Generated
...
```

E.2 Evaluation Metrics

```
Initial Portfolio Value : 100000
Final Portfolio Value   : Generated during execution
Cumulative Return       : Generated during execution
Average Reward          : Generated during execution
Volatility              : Generated during execution
```

Sharpe Ratio : Generated during execution

Maximum Drawdown : Generated during execution

E.3 Performance Evaluation

The following metrics are calculated from the actual execution of the program:

```
total_return = (
    final_value - initial_value
) / initial_value

volatility = np.std(
    portfolio_returns,
    ddof=1
)

sharpe_ratio = (
    np.mean(portfolio_returns) - risk_free_rate
) / volatility

peak = np.maximum.accumulate(
    portfolio_values
)

drawdown = (
    portfolio_values - peak
) / peak

maximum_drawdown = drawdown.min()
```

Appendix F – Ethics / Institutional Approval

F.1 Responsible Financial Decision Support

The developed system provides portfolio-management recommendations using historical market information and reinforcement learning. It is intended as an academic decision-support prototype and does not guarantee investment returns.

F.2 Risk and Human Oversight

RL Recommendation

↓

Risk Analysis

↓

Performance Evaluation

↓

Human Review

↓

Final Decision

The final investment decision should remain under human control.

F.3 Data Privacy and Security

User portfolio information should be processed securely. Sensitive information should not be unnecessarily stored, displayed in logs, or exposed through model outputs.

Appendix G – Project Meeting / Progress Records

Phase	Activity	Module
1	Requirement Analysis	All Modules
2	User Management Design	Module 1
3	Portfolio Management Design	Module 1
4	Market Data Processing	Module 2
5	Risk Analysis Implementation	Module 2
6	RL Environment Design	Module 3

7| State and Action Design| Module 3

8| Policy Network Development| Module 3

9| Policy Gradient Training| Module 3

10| Model Evaluation| Module 3

11| Decision-Support Integration| Module 3

12| Testing and Documentation| All Modules

Appendix H – User/Industry Feedback

H.1 System Evaluation Areas

Evaluation Area| System Output

User Management| User and portfolio information

Portfolio Management| Asset allocation information

Market Analysis| Processed market information

Risk Analysis| Volatility, Sharpe ratio and drawdown

RL Recommendation| Portfolio allocation recommendation

Performance Analysis| Portfolio performance metrics

Decision Support| Risk-aware investment recommendation

External user or industry feedback should be included only when actual feedback has been collected.

Appendix I – Publications / Patents / Competitions / Product Outcomes

I.1 Project Outcome

Project:

Financial Portfolio Management and Risk Analysis Using

Policy Gradient Reinforcement Learning

Academic Outcome:

Reinforcement Learning based financial portfolio management and risk-analysis prototype.

Module 1:

User & Portfolio Management

Module 2:

Market Data & Risk Analysis

Module 3:

Policy Gradient Reinforcement Learning & Decision Support

No publication, patent, competition award, or commercial-product claim is included without verified evidence.

Appendix J – Individual Contribution Evidence

J.1 Module 1 – User & Portfolio Management

Contribution Evidence:

• User-management implementation

• Portfolio creation and management

• Asset allocation handling

• Portfolio information processing

• Module testing

J.2 Module 2 – Market Data & Risk Analysis

Contribution Evidence:

• Market-data processing

• Asset-return calculation

• Portfolio-return calculation

• Volatility calculation

• Sharpe-ratio calculation

• Maximum-drawdown calculation

• Risk visualization

J.3 Module 3 – Policy Gradient Reinforcement Learning & Decision Support

Contribution Evidence:

• RL environment development

• State representation

• Action selection

• Reward-function implementation

• Policy-network development

• Policy-gradient training

• Policy update

• Model evaluation

• Decision-support implementation

J.4 Contribution Evidence Table

Contribution Area	Evidence
User Management	Module 1 implementation
Portfolio Management	Portfolio-management implementation
Market Data	Market-data processing module
Risk Analysis	Risk-analysis calculations
RL Environment	Portfolio environment implementation
Policy Gradient	Policy network and agent
Model Training	Training implementation
Evaluation	Evaluation and metrics
Visualization	Performance and risk graphs
Documentation	Project report and technical documentation

Note: Student names and contribution percentages should be entered only according to the actual project team and verified contribution evidence.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

The mapping below follows the original template. Evidence locations must be updated after the final project files are incorporated.

Outcome	Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	Financial portfolio decision problem under uncertainty	SO1	Ch. 1, Ch. 2
Engineering design under constraints	State/action/reward and portfolio-risk constraints	SO2	Ch. 3
Written/oral technical communication	Structured report and decision-support explanation	SO3	Whole report + Viva
Ethics and professional responsibility	Responsible financial decision-support framing	SO4	Ch. 3.7, Ch. 5
Teamwork and leadership	Roles/contribution evidence	SO5	Ch. 4.8 + Appendix J
Experimentation, analysis, engineering judgment	Test plan and financial/RL evaluation metrics	SO6	Ch. 4.3–4.6
Independent acquisition of new knowledge	Policy-gradient and risk-analysis learning	SO7	Ch. 5.3
Standards	Applicable academic/data/software practices	SO2	Ch. 3.2
Sustainability and societal/environmental impact	Computational cost and financial harm considerations	SO2/SO4	Ch. 3.7
Risk assessment and mitigation	Overfitting, leakage, market-regime and allocation risks	SO2/SO4	Ch. 3.7
Security considerations	Portfolio/user data protection	Relevant SO	Ch. 3.7
Integrated/system-level engineering	Three-module integrated architecture	SO1/SO2	Ch. 3.5
Project planning and management	Milestones, roles and budget	SO5	Ch. 4.8
Individual contribution	Verified student responsibilities	SO1–SO7	Contribution Statement + Ch. 4.8 + Appendix J